

```
> 1 && 2
[1] TRUE

> TRUE | FALSE
[1] TRUE

> TRUE || FALSE
[1] TRUE
```

If in doubt, you can use the help ‘?’ operator or the *help()* function to know more about the operation. For example:

```
> ?TRUE
```

Logical Vectors

Description:

Create or test for objects of type “logical”, and the basic logical constants.

Usage:

```
TRUE
FALSE
T; F

logical(length = 0)
as.logical(x, ...)
is.logical(x)
```

Arguments:

...

```
> help("+")
```

Arithmetic Operators

Description:

These unary and binary operators perform arithmetic on numeric or complex vectors (or objects which can be coerced to them).

Usage:

```
+ x
- x
x + y
x - y
x * y
x / y
```

```
x ^ y
x %% y
x %/% y
```

Arguments:

x, y: numeric or complex vectors or objects which can be coerced to such, or other objects for which methods have been written.

Details:

...

The order of precedence for the various R operators from the highest to lowest is given in the following table:

Operator	Description
:: :::	access variables in a name space
\$ @	component / slot extraction
[[[	indexing
^	exponentiation (right to left)
- +	unary minus and plus
:	sequence operator
%any%	special operators (including %% and %/%)
* /	multiply, divide
+ -	(binary) add, subtract
< > <= >=	ordering and comparison
= !	negation
! !	negation
& &&	and
	or
~	as in formulae
-> ->>	rightwards assignment
<- <<-	assignment (right to left)
=	assignment (right to left)
?	help (unary and binary)

Function

A function is also an object in R that takes objects as input and returns an output object. The syntax is as follows:

```
function (argument1, argument2, ...)
```

Everything in R is an object, and unary operator functions have an equivalent form. For example:

```
> 1 == 2
[1] FALSE

> -3
[1] -3
```